Examples

```
function ContainsDuplicates(A[],n)
    for i = 2 to n do
        for j = 1 to i − 1 do
            if A[i] = A[j] then
                return true
            end if
        end for
    end for
    return false
end function
```

Example 1. Determine the asymptotic complexity of the given function.

```
function Func1(n)
    x = 0
    for i = 1 to n do
        for j = 1 to √i do
            x = x + (i − j)
        end for
    end for
    return x
end function
```

Let $T(n)$ be the running time of `Func1` with input n. We can express the running time of the function as the summation,

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{\sqrt{i}} c = \sum_{i=1}^{n} c\sqrt{i}$$

There is no known convenient closed form expression for this summation and therefore we resort to bounding this summation. We hope to find some function $f(n)$ such that

$$c_1 f(n) \leq \sum_{i=1}^{n} c\sqrt{i} \leq c_2 f(n).$$

Our process for doing this varies depending upon the summation being analyzed. For this summation we shall bound the summation above by replacing every term of the summation with the largest term, that is

$$T(n) = \sum_{i=1}^{n} c\sqrt{i} \leq \sum_{i=1}^{n} c\sqrt{n}.$$

Having bounded the summation above in this manner it becomes much easier to determine a closed form expression. Notice that $c\sqrt{n}$ no longer depends upon the summation index i and the sumation reduces to

$$T(n) \leq \underbrace{c\sqrt{n} + c\sqrt{n} + \cdots + c\sqrt{n}}_{n \text{ terms}} = n(c\sqrt{n}) = cn^{1.5}$$

We can therefore conclude that $\boxed{T(n) \in O(n^{1.5}).}$

We (ideally) now want to show that $T(n) \in \Omega(n^{1.5})$ – doing this typically requires splitting the summation into two parts,

$$T(n) = \sum_{i=1}^{n} c\sqrt{i} = \sum_{i=1}^{\frac{n}{2}} c\sqrt{i} + \sum_{i=\frac{n}{2}+1}^{n} c\sqrt{i}$$

we then bound this below by observing that the first summation (the one that has the smaller terms) is positive, therefore

$$T(n) = \sum_{i=1}^{\frac{n}{2}} c\sqrt{i} + \sum_{i=\frac{n}{2}+1}^{n} c\sqrt{i} \geq \sum_{i=\frac{n}{2}+1}^{n} c\sqrt{i}$$

We further observe that since the remaining summation is an increasing function of the summation index that every term is greater than $c\sqrt{\frac{n}{2}}$, so

$$T(n) \geq \sum_{i=\frac{n}{2}+1}^{n} c\sqrt{i} \geq \sum_{i=\frac{n}{2}+1}^{n} c\sqrt{\frac{n}{2}} = \left(n - \left(\frac{n}{2}+1\right)+1\right) c\sqrt{\frac{n}{2}} = \frac{n}{2} c\sqrt{\frac{n}{2}}$$

Which can be simplified to become

$$T(n) \geq \frac{c}{2\sqrt{2}} n^{1.5},$$

Therefore, we have shown that $\boxed{T(n) \in \Omega(n^{1.5}).}$
Since $T(n) \in O(n^{1.5})$ and $T(n) \in \Omega(n^{1.5})$ we can conclude that

$$\boxed{T(n) \in \Theta(n^{1.5})}$$

In most future examples we will show significantly less explanation since we assume that the reader has seen these things before and can adopt the methodology shown here appropriately.

Example 2. Determine the asymptotic complexity of the given function.

```
function Func2(n)
    x = 0
    for i = 1 to √n do
        for j = 1 to ⌊√n⌋ − i do
            x = x + (i − j)
        end for
    end for
    return x
end function
```

Let $T(n)$ be the running time of `Func2` with input n.

$$T(n) = \sum_{i=1}^{\sqrt{n}} c(\sqrt{n} - i)$$

First we shall bound this summation above,

$$T(n) = \sum_{i=1}^{\sqrt{n}} c(\sqrt{n} - i) \leq \sum_{i=1}^{\sqrt{n}} c(\sqrt{n} - 1) \leq \sum_{i=1}^{\sqrt{n}} c(\sqrt{n}) = \sqrt{n}c\sqrt{n} = cn$$

Therefore, $T(n) \in O(n)$.
We now bound the summation below,

$$\begin{aligned} T(n) = \sum_{i=1}^{\sqrt{n}} c(\sqrt{n} - i) &= \sum_{i=1}^{\sqrt{n}/2} c(\sqrt{n} - i) + \sum_{i=\sqrt{n}/2+1}^{\sqrt{n}} c(\sqrt{n} - i) \\ &\geq \sum_{i=1}^{\sqrt{n}/2} c(\sqrt{n} - i) \\ &\geq \sum_{i=1}^{\sqrt{n}/2} c\left(\sqrt{n} - \frac{\sqrt{n}}{2}\right) \\ &= \frac{\sqrt{n}}{2} c \frac{\sqrt{n}}{2} = \frac{c}{4} n \end{aligned}$$

Therefore, $T(n) \in \Omega(n)$.
Thus,

$$\boxed{T(n) \in \Theta(n)}.$$

Example 3. Determine the asymptotic complexity of the given function.

```
function Func3(n)
    x = 0
    i = 7
    while i ≤ n do
        x = x + i
        i = i + 3
    end while
    return x
end function
```

Let $T(n)$ be the running time of `Func3` with input of size n.
To analyze the running time of a While Loop we will need to find some means of determining the number of times that the while loop will execute. One way of doing this is to create a table to characterize how the loop variable evolves,

Iteration	Value of i
0	7
1	10
2	13
3	16
$\vdots$	$\vdots$
k	$7+3k$

The loop will terminate when $i > n$ – that is

$$7 + 3k > n \qquad \Longrightarrow \qquad k > \frac{n-7}{3}.$$

A reasonable approximation that we shall make is that rather than finding the first value of k to satisfy the above inequality, we shall instead determine when $i = n$. Note that this will only **approximate** the number of iterations of the while loop and does not find the **exact** number of iterations. For certain problems and applications you may wish to perform a more careful analysis, but we will not be doing so here. Therefore we assume that the while executes $k = \frac{n-7}{3}$ times. Since the cost of each iteration of the while loop is constant, the total running time is

$$T(n) = \frac{n-7}{3}c$$

Therefore,

$$\boxed{T(n) \in \Theta(n).}$$

Example 4. Determine the asymptotic complexity of the given function.

```
function Func4(n)
    x = 0
    i = 1
    while i ≤ n do
        x = x + i
        i = 2i
    end while
    return x
end function
```

Let $T(n)$ be the running time of `Func4` with input of size n.
We first determine the number of iterations of the while loop,

Iteration	Value of i
0	1
1	2
2	4
3	8
$\vdots$	$\vdots$
k	2^k

We approximate the number of iterations by solving

$$2^k = n \qquad \Longrightarrow \qquad k = \log_2(n).$$

The cost of each iteration is constant so

$$T(n) = c\log_2(n)$$

Therefore,

$$\boxed{T(n) \in \Theta(\log(n)).}$$

Example 5. Determine the asymptotic complexity of the given function.

```
function Func5(n)
    x = 0
    i = 7
    while i ≤ n do
        x = x + i
        i = 2i
    end while
    return x
end function
```

Let $T(n)$ be the running time of `Func5` with input of size n.
We first determine the number of iterations of the while loop,

Iteration	Value of i
0	7
1	$7 \cdot 2$
2	$7 \cdot 2^2$
3	$7 \cdot 2^3$
$\vdots$	$\vdots$
k	$7 \cdot 2^k$

We approximate the number of iterations by solving

$$7 \cdot 2^k = n \qquad \Longrightarrow \qquad k = \log_2(n/7) = \log_2(n) - \log_2(7) \approx \log_2(n).$$

The cost of each iteration is constant so

$$T(n) \approx c \log_2(n)$$

Therefore,

$$\boxed{T(n) \in \Theta(\log(n)).}$$

Example 6. Determine the asymptotic complexity of the given function.

```
function Func6(n)
    x = 0
    i = 7
    while i ≤ n do
        x = x + i
        i = 3i
    end while
    return x
end function
```

Let $T(n)$ be the running time of `Func6` with input of size n.
We first determine the number of iterations of the while loop,

Iteration	Value of i
0	1
1	3
2	9
3	27
$\vdots$	$\vdots$
k	3^k

We approximate the number of iterations by solving

$$3^k = n \qquad \Longrightarrow \qquad k = \log_3(n).$$

The cost of each iteration is constant so

$$T(n) = c\log_3(n)$$

Therefore,

$$\boxed{T(n) \in \Theta(\log(n)).}$$

Example 7. Determine the asymptotic complexity of the given function.

```
1: function Func7(n)
2:     x = 0
3:     for i = 1 to n do
4:         j = 1
5:         while j ≤ n do
6:             x = x + (i − j)
7:             j = 2j
8:         end while
9:     end for
10:    return x
11: end function
```

Let $T(n)$ be the running time of `Func7` with input n.

We begin by analyze the body of the outer loop (lines 4-8). To determine the number of iterations of the while loop we create an iteration table,

Iteration	Value of j
0	1
1	2
2	4
3	8
$\vdots$	$\vdots$
k	2^k

We approximate the number of iterations by solving

$$2^k = n \qquad \Longrightarrow \qquad k = \log_2(n).$$

The cost of each iteration of the while loop is constant the running time of lines 4-8 is $c\log_2(n)$. Now,

$$T(n) = \sum_{i=1}^{n} c\log_2(n) = nc\log_2(n)$$

Example 8. Determine the asymptotic complexity of the given function.

```
1: function Func8(n)
2:     x = 0
3:     for i = 1 to n do
4:         j = 1
5:         while j ≤ i do
6:             x = x + (i − j)
7:             j = 2j
8:         end while
9:     end for
10:    return x
11: end function
```

Let $T(n)$ be the running time of `Func8` with input n.
We begin by analyze the body of the outer loop (lines 4-8). To determine the number of iterations of the while loop we create an iteration table,

Iteration	Value of j
0	1
1	2
2	4
3	8
$\vdots$	$\vdots$
k	2^k

We approximate the number of iterations by solving

$$2^k = i \qquad \Longrightarrow \qquad k = \log_2(i).$$

The cost of each iteration of the while loop is constant the running time of lines 4-8 is $c\log_2(i)$. Notice that the running time of line 4-8 now depends upon i, so

$$T(n) = \sum_{i=1}^{n} c\log_2(i)$$

must be bounded.
Bounding above we have,

$$T(n) = \sum_{i=1}^{n} c\log_2(i) \leq \sum_{i=1}^{n} c\log_2(n) = nc\log_2(n)$$

Bounding below we have,

$$T(n) = \sum_{i=1}^{n} c\log_2(i) \geq \sum_{i=n/2+1}^{n} c\log_2(i) \geq \sum_{i=n/2+1}^{n} c\log_2\left(\frac{n}{2}\right) = \frac{n}{2}c\log_2\left(\frac{n}{2}\right)$$

Therefore,

$$\boxed{T(n) \in \Theta(n\log(n)).}$$

Example 9. Determine the asymptotic complexity of the given function.

```
function Func9(n)
    x = 0
    for i = 1 to n do
        for j = 1 to i do
            for k = j to i do
                x = x + (ki − j)
            end for
        end for
    end for
    return x
end function
```

Let $T(n)$ be the running time of `Func9` with input n.
We have that

$$\begin{aligned}
T(n) = \sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=j}^{i} c &= \sum_{i=1}^{n}\sum_{j=1}^{i} c(i-j+1) \\
&= c\sum_{i=1}^{n}\left[\sum_{j=1}^{i}(i+1) - \sum_{j=1}^{i} j\right] \\
&= c\sum_{i=1}^{n}\left[i(i+1) - \frac{i(i+1)}{2}\right] \\
&= c\sum_{i=1}^{n}\left[i^2 + i - \frac{i^2}{2} - \frac{i}{2}\right] \\
&= \frac{c}{2}\sum_{i=1}^{n}\left[i^2 + i\right] \\
&= \frac{c}{2}\left[\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}\right]
\end{aligned}$$

Therefore,

$$\boxed{T(n) \in \Theta(n^3).}$$

Alternatively, we could bound the summation

$$
\begin{aligned}
T(n) = \sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=j}^{i} c = \sum_{i=1}^{n}\sum_{j=1}^{i} c(i-j+1) \\
&\leq \sum_{i=1}^{n}\sum_{j=1}^{i} ci \leq \sum_{i=1}^{n} ci^2 \leq \sum_{i=1}^{n} cn^2 = cn^3 \\
&\geq \sum_{i=1}^{n}\sum_{j=1}^{i/2} c(i-j+1) \geq \sum_{i=1}^{n}\sum_{j=1}^{i/2} c(i-(i/2)) \geq \sum_{i=1}^{n}\sum_{j=1}^{i/2} \frac{ci}{2} \\
&\geq \sum_{i=1}^{n} \frac{ci^2}{4} \geq \sum_{i=n/2+1}^{n} \frac{ci^2}{4} \geq \sum_{i=n/2+1}^{n} \frac{c\left(\frac{n}{2}\right)^2}{4} \\
&\geq \frac{n}{2} \cdot \frac{cn^2}{16} = \frac{cn^3}{32}
\end{aligned}
$$

Therefore,

$$\boxed{T(n) \in \Theta(n^3).}$$

Example 10. Determine the asymptotic complexity of the given function.

```
function Func10(n)
    x = 0
    i = 1
    while i ≤ n do
        for j = 1 to i do
            x = x + (i − j)
        end for
        i = 2i
    end while
    return x
end function
```

Let $T(n)$ be the running time of `Func10` with input n. We begin by analyzing the inner most loop which takes ci time. We then analyze the outer loop. To analyze the while loop we construct an iteration table,

Iteration	Value of i
0	1
1	2
2	4
3	8
$\vdots$	$\vdots$
k	2^k

We approximate the number of iterations by solving

$$2^k = n \qquad \Longrightarrow \qquad k = \log_2(n).$$

Now, the cost of each iteration of the while loop is ci which presents us with a new problem. Conveniently, our iteration table tells us every value that i takes as the code executes and we can therefore express $T(n)$ as the summation

$$T(n) = c + 2c + 4c + 8c + \cdots + cn.$$

We can express this in some more familiar notation,

$$T(n) = \sum_{\substack{\text{Values} \\ \text{of } i}} ci$$

The major reason that this summation is difficult to analyze is that the summation variable i does not increase in fixed increments – instead it increases exponentially. Thankfully, we already know how to re-express i in terms of another variable, k, that does increase in fixed increments. We shall perform this substitution and analyze the resulting summation,

$$T(n) = \sum_{\substack{\text{Values} \\ \text{of } i}} ci = \sum_{k=0}^{\log_2(n)} c2^k$$

We now have a familiar summation that we have a closed form expression for,

$$T(n) = \sum_{k=0}^{\log_2(n)} c2^k = c\frac{2^{\log_2(n)+1} - 1}{2-1} = c(2 \cdot 2^{\log_2(n)} - 1) = c(2n-1)$$

Therefore,

$$\boxed{T(n) \in \Theta(n).}$$

Alternatively, we could have bounded the original summation – this bounding technique works for geometric summations and is the recommended approach. So,

$$\begin{aligned}
T(n) &= c + 2c + 4c + \cdots + \frac{cn}{4} + \frac{cn}{2} + cn \\
&= cn + \frac{cn}{2} + \frac{cn}{4} + \cdots + 4c + 2c + c \\
&= cn\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{4}{n} + \frac{2}{n} + \frac{1}{n}\right) \\
&\leq cn\left(1 + \frac{1}{2} + \frac{1}{4} + \cdots\right) \\
&\leq cn\left(\frac{1}{1-\frac{1}{2}}\right) = 2cn \\
&\geq cn
\end{aligned}$$

Therefore,

$$\boxed{T(n) \in \Theta(n).}$$

Example 11. Determine the asymptotic complexity of the given function.

```
1:  function Func11(n)
2:      x = 0
3:      i = 1
4:      while i ≤ n do
5:          j = 1
6:          while j ≤ 2i do
7:              x = x + (i − j)
8:              j = j + 4
9:          end while
10:         i = 3i
11:     end while
12:     return x
13: end function
```

Let $T(n)$ be the running time of `Func11` with input n.
We begin by analyze the body of the outer loop (lines 5-10). To determine the number of iterations of the inner while loop we create an iteration table,

Iteration	Value of j
0	1
1	5
2	9
3	13
$\vdots$	$\vdots$
k	$1+4k$

We approximate the number of iterations by solving

$$1+4k=2i \qquad \Longrightarrow \qquad k=\frac{2i-1}{4}\approx\frac{i}{2}$$

Since the cost of each iteration of the inner while loop is constant, the cost of running lines 5-10 is $\frac{c'i}{2}$. If we let $c=\frac{c'}{2}$ then the cost of each iteration of lines 5-10 is ci. To analyze the outer while loop (lines 4-11) we construct another iteration table,

Iteration	Value of i
0	1
1	3
2	3^2
3	3^3
$\vdots$	$\vdots$
γ	3^γ

We approximate the number of iterations by solving

$$3^\gamma=n \qquad \Longrightarrow \qquad \gamma=\log_3(n)$$

Now since the cost of each run of the outer while loop is ci, the total running time is

$$T(n) = c + 3c + 9c + \cdots + \frac{cn}{3} + cn$$

Since this is a geometric summation we bound it above by an infinite geometric series,

$$\begin{aligned}
T(n) &= c + 3c + 9c + \cdots + \frac{cn}{3} + cn \\
&= cn + \frac{cn}{3} + \frac{cn}{9} + \cdots + 3c + c \\
&= cn\left(1 + \frac{1}{3} + \frac{1}{9} + \cdots + \frac{3}{n} + \frac{1}{n}\right) \\
&\leq cn\left(1 + \frac{1}{3} + \frac{1}{9} + \cdots\right) \\
&\leq cn\left(\frac{1}{1 - \frac{1}{3}}\right) = \frac{3cn}{2}
\end{aligned}$$

and bound it below trivially

$$T(n) = cn + \frac{cn}{3} + \frac{cn}{9} + \cdots + 3c + c \geq cn.$$

Therefore,

$$\boxed{T(n) \in \Theta(n).}$$

Example 12. Determine the asymptotic complexity of the given function.

```
1:  function Func12(n)
2:      x = 0
3:      i = 1
4:      while i ≤ n do
5:          for j = 1 to ⌊n/i⌋ do
6:              x = x + (i − j)
7:          end for
8:          i = 2i
9:      end while
10:     return x
11: end function
```

Let $T(n)$ be the running time of `Func12` with input n.
We begin by analyzing the running time of the body of the outer loop. The running time of lines 5-8 is $\frac{cn}{i}$. To analyze the running time of the while loop we construct an iteration table,

Iteration	Value of i
0	1
1	2
2	2^2
3	2^3
$\vdots$	$\vdots$
ξ	2^ξ

We approximate the number of iterations by solving

$$2^\xi = n \qquad \Longrightarrow \qquad \xi = \log_2(n).$$

The total running time can be expressed as

$$\begin{aligned} T(n) &= \frac{cn}{1} + \frac{cn}{2} + \frac{cn}{4} + \cdots + \frac{cn}{\frac{n}{2}} + \frac{cn}{n} \\ &= cn + \frac{cn}{2} + \frac{cn}{4} + \cdots + 2c + c \end{aligned}$$

We analyzed this exact summation in the previous problem and showed that

$$\boxed{T(n) \in \Theta(n).}$$

Example 13. Determine the asymptotic complexity of the given function.

```
function Func13(n)
    x = 0
    i = 1
    for i = 2n to 3n^2 + 5n do
        for j = 1 to i^3 + i^2 do
            x = x + (i − j)
        end for
    end for
    return x
end function
```

Let $T(n)$ be the running time of `Func13` with input n.
We express $T(n)$ as a summation,

$$T(n) = \sum_{i=2n}^{3n^2+5n} \sum_{j=1}^{i^3+i^2} c = \sum_{i=2n}^{3n^2+5n} c(i^3 + i^2)$$

We now proceed by bounding this summation.

Bounding above

$$\begin{aligned} T(n) = \sum_{i=2n}^{3n^2+5n} c(i^3 + i^2) &\leq \sum_{i=1}^{8n^2} 2ci^3 \\ &\leq \sum_{i=1}^{8n^2} 2c(8n^2)^3 \\ &\leq 2c(8n^2)^4 = 2^{13}cn^8 \end{aligned}$$

So,

$$\boxed{T(n) \in O(n^8).}$$

Bounding below

$$\begin{aligned} T(n) = \sum_{i=2n}^{3n^2+5n} c(i^3 + i^2) &\geq \sum_{i=2n^2}^{3n^2} ci^3 \\ &\geq \sum_{i=2n^2}^{3n^2} c(n^2)^3 \\ &\geq (n^2 + 1)cn^6 \geq cn^8 \end{aligned}$$

So,

$$\boxed{T(n) \in \Omega(n^8).}$$

Therefore,

$$\boxed{T(n) \in \Theta(n^8).}$$